

Procedimento Operacional Padrão para Acesso, Extração e Análise de Dados em Bancos de Dados Aplicados à Engenharia de Produção

Dayvid Wesley Pereira Martins (UFG)

Júlia Gabriela Brito Ferreira (UFG)

Samuel Lorenzo Schirmbeck (UFG)



A crescente digitalização dos ambientes produtivos e a consolidação do paradigma da Indústria 4.0 têm gerado volumes expressivos de dados nos sistemas industriais, tornando indispensável o desenvolvimento de procedimentos estruturados para acesso, extração e análise dessas informações. O presente trabalho propõe um Procedimento Operacional Padrão (POP) voltado ao engenheiro de produção, contemplando desde a modelagem conceitual dos dados até a extração de indicadores de desempenho via consultas SQL em bancos de dados relacionais. A metodologia adotada é de natureza aplicada, com abordagem qualitativa e procedimento técnico-descritivo. A fundamentação teórica abrange Sistemas de Informação Gerencial (SIG), arquitetura de software, modelagem de dados, normalização relacional, UML e linguagem SQL. O POP proposto estrutura sete etapas operacionais: identificação da fonte, compreensão da estrutura, modelagem, conexão, extração, organização e análise. Contribuindo para a autonomia analítica do engenheiro de produção na geração de conhecimento a partir de dados primários.

Palavras-chave: Banco de dados relacional; Banco de dados não relacional; MongoDB; PostgreSQL; Procedimento Operacional Padrão; SQL; UML; Modelagem de dados; Sistemas de Informação Gerencial.

1. Introdução

Nas últimas décadas, o avanço das tecnologias digitais tem promovido transformações significativas nos sistemas produtivos, consolidando o paradigma da Indústria 4.0. Esse modelo industrial caracteriza-se pela integração entre sistemas físicos e digitais, bem como pela utilização de tecnologias como Internet das Coisas (IoT), inteligência artificial e análise avançada de dados, permitindo a coleta e o processamento de grandes volumes de informações geradas nos ambientes produtivos (Piispanen; Hentunen, 2026). Nesse contexto, a digitalização dos processos industriais tem se tornado um fator estratégico para a melhoria da eficiência operacional e da tomada de decisão nas organizações.

A crescente digitalização das operações produtivas resulta em um aumento expressivo da quantidade de dados gerados pelas empresas. Sensores, sistemas de planejamento empresarial e plataformas de monitoramento industrial produzem continuamente grandes volumes de informações que podem ser utilizados para compreender o desempenho dos sistemas produtivos e apoiar decisões gerenciais (Corallo et al., 2022). Nesse cenário, a análise de dados em larga escala, conhecida como Big Data Analytics, tem se consolidado como um importante recurso tecnológico para a transformação digital da manufatura, possibilitando a identificação de padrões operacionais e a geração de conhecimento a partir de dados industriais (Wang et al., 2022).

Além disso, a incorporação de técnicas de inteligência artificial e aprendizado de máquina amplia a capacidade de análise desses dados, permitindo o desenvolvimento de sistemas capazes de prever falhas, otimizar processos e apoiar a tomada de decisão em ambientes produtivos complexos (Akram et al., 2026). Entretanto, apesar do potencial dessas tecnologias, muitas organizações ainda enfrentam desafios relacionados à integração, gestão e análise eficiente dos dados provenientes de diferentes sistemas industriais (Corallo et al., 2022).

Diante desse contexto, torna-se fundamental o desenvolvimento de métodos estruturados que possibilitem acessar, organizar e analisar dados provenientes de sistemas industriais. Nesse sentido, a Engenharia de Produção desempenha um papel relevante ao integrar conhecimentos de gestão, tecnologia da informação e análise de dados para apoiar decisões baseadas em informações confiáveis. Assim, este trabalho tem como objetivo desenvolver um Procedimento Operacional Padrão (POP) para acesso, extração e análise de dados em bancos de dados aplicados à Engenharia de Produção, contribuindo para a

sistematização do processo de geração de informações para a tomada de decisão em ambientes produtivos.

2. Fundamentação Teórica

2.1 Sistemas de Informação Gerencial

Os sistemas de informação desempenham papel fundamental no suporte à gestão organizacional e à tomada de decisão em ambientes produtivos. De maneira geral, sistemas de informação podem ser definidos como conjuntos integrados de componentes responsáveis por coletar, processar, armazenar e disseminar informações que apoiam as atividades operacionais e gerenciais das organizações (Laudon; Laudon, 2020).

No contexto da engenharia de produção, os sistemas de informação são amplamente utilizados para monitorar processos produtivos, gerenciar recursos e acompanhar indicadores de desempenho organizacional. A utilização dessas ferramentas permite que gestores tenham acesso a informações estruturadas e atualizadas sobre o funcionamento dos sistemas produtivos, contribuindo para a melhoria do planejamento, controle e tomada de decisão (O'Brien; Marakas, 2013). Esses sistemas podem ser classificados em diferentes tipos, de acordo com o nível organizacional e o propósito ao qual servem: Sistemas de Processamento de Transações (TPS), responsáveis pelo registro das operações rotineiras; Sistemas de Informação Gerencial (SIG), voltados ao suporte a decisões estruturadas de nível tático; Sistemas de Suporte à Decisão (DSS), orientados a análises complexas e semiestruturadas; e Sistemas de Informação Executiva (EIS), destinados ao nível estratégico. Em ambientes industriais, destacam-se ainda os sistemas ERP (Enterprise Resource Planning), MES (Manufacturing Execution System) e SCADA (Supervisory Control and Data Acquisition), que integram dados de diferentes camadas da operação (Laudon; Laudon, 2020).

Um conceito central para compreender o valor dos sistemas de informação é o pensamento sistêmico, que propõe analisar as organizações não como conjuntos de partes isoladas, mas como sistemas interdependentes, nos quais a alteração de um elemento repercute sobre os demais (Senge, 1990). Aplicado à Engenharia de Produção, o pensamento sistêmico orienta o engenheiro a compreender que os dados gerados em cada etapa do processo produtivo fazem parte de um sistema mais amplo de informações, cuja integração é necessária para a tomada de decisão eficaz.

No âmbito dos sistemas de informação, é fundamental compreender a hierarquia que transforma dados brutos em suporte à decisão. Dados são fatos isolados, ainda sem contexto ou interpretação — como um registro de tempo de ciclo armazenado em uma tabela. Quando

organizados, filtrados e contextualizados, esses dados tornam-se informação, capaz de descrever o desempenho de uma linha de produção. Ao serem analisados, comparados com metas e interpretados pelo engenheiro de produção, a informação transforma-se em conhecimento, que por sua vez fundamenta decisões gerenciais. Esse ciclo — dado → informação → conhecimento → decisão — representa a essência do valor gerado pelos sistemas de informação em ambientes produtivos (Laudon; Laudon, 2020; Davenport, 2014), e é o ciclo que o POP proposto neste trabalho busca operacionalizar de forma sistemática e reproduzível.

Com o avanço da transformação digital, os sistemas de informação passaram a integrar tecnologias associadas à Indústria 4.0, como sensores industriais, Internet das Coisas (IoT) e plataformas de análise de dados. Esse cenário possibilita maior capacidade de coleta e processamento de dados provenientes de ambientes produtivos, permitindo o desenvolvimento de sistemas mais inteligentes e orientados por dados (Piispanen; Hentunen, 2026).

Nesse contexto, a manufatura inteligente tem se consolidado como um novo paradigma produtivo baseado na integração entre tecnologias digitais e sistemas industriais. Essa abordagem possibilita maior visibilidade sobre os processos produtivos e maior capacidade de análise de dados para suporte à tomada de decisão (Javaid et al., 2024). Além disso, organizações orientadas por dados apresentam maior capacidade de gerar conhecimento a partir de informações operacionais, utilizando essas informações para melhorar o desempenho organizacional e apoiar decisões estratégicas (Davenport, 2014).

2.2 Arquitetura de Software em Sistemas Industriais

A arquitetura de software representa a estrutura fundamental de um sistema computacional, definindo seus componentes, suas interações e os princípios que orientam seu desenvolvimento e evolução ao longo do tempo (Bass; Clements; Kazman, 2013). Em sistemas corporativos e industriais, uma arquitetura bem definida é essencial para garantir escalabilidade, manutenção e integração entre diferentes aplicações.

Entre os modelos arquiteturais mais utilizados destaca-se a arquitetura em camadas, na qual o sistema é organizado em diferentes níveis de responsabilidade: camada de apresentação (interface do usuário), camada de lógica de negócio (regras e processamento) e camada de dados (armazenamento e persistência). Essa organização facilita a manutenção dos sistemas e permite maior modularidade no desenvolvimento de aplicações (Sommerville, 2019). Nessa estrutura, o banco de dados ocupa o papel de camada de infraestrutura desacoplada,

responsável pelo armazenamento persistente e confiável dos dados, acessível pelas camadas superiores por meio de interfaces padronizadas. Esse desacoplamento garante que mudanças na interface ou na lógica de negócio não impactem diretamente a integridade dos dados armazenados.

Outro modelo amplamente utilizado é a arquitetura Model–View–Controller (MVC), que separa a representação dos dados, a lógica de negócio e a interface de interação com o usuário. Essa separação contribui para o desenvolvimento de sistemas mais organizados e flexíveis (Pressman; Maxim, 2020). Abordagens mais modernas, como a Clean Architecture proposta por Martin (2017), reforçam o princípio de que as regras de negócio devem ser independentes de frameworks, interfaces e, especialmente, do banco de dados, posicionando este último como um detalhe de infraestrutura que pode ser substituído sem impacto nas regras centrais do sistema.

Nos últimos anos, novas abordagens arquitetônicas passaram a ser adotadas em sistemas industriais, incluindo arquiteturas orientadas a serviços e modelos baseados em microsserviços. Essas arquiteturas buscam promover maior desacoplamento entre os componentes do sistema, permitindo maior flexibilidade e integração entre diferentes plataformas tecnológicas (Martin, 2017).

Em ambientes industriais digitalizados, a arquitetura de software também desempenha papel importante na integração entre sistemas corporativos, como sistemas de planejamento empresarial (ERP), sistemas de execução da manufatura (MES) e plataformas de monitoramento de produção. Essa integração permite o compartilhamento de dados entre diferentes aplicações, favorecendo o desenvolvimento de ambientes produtivos mais conectados e inteligentes (Xu; Xu; Li, 2024).

2.3 Bancos de Dados em Sistemas de Produção

Os bancos de dados representam a principal infraestrutura para armazenamento, organização e recuperação de dados em sistemas computacionais. De acordo com Silberschatz, Korth e Sudarshan (2020), um banco de dados pode ser definido como um conjunto estruturado de dados relacionados, armazenados de forma organizada para permitir acesso eficiente e manipulação das informações.

Historicamente, os bancos de dados evoluíram ao longo de décadas, acompanhando as demandas crescentes das organizações. Na década de 1960, os primeiros sistemas de gerenciamento utilizavam arquivos sequenciais e estruturas hierárquicas, como o IMS da IBM. Em 1970, Edgar F. Codd propôs o modelo relacional, introduzindo a organização dos

dados em tabelas inter-relacionadas e as bases para o desenvolvimento da linguagem SQL (Codd, 1970). Nas décadas de 1980 e 1990, consolidaram-se os grandes sistemas gerenciadores de banco de dados (SGBDs) relacionais, como Oracle, PostgreSQL e MySQL. Nos anos 2000, o crescimento do volume de dados gerado pela internet impulsionou o movimento NoSQL, com soluções como MongoDB, Cassandra e Redis, projetadas para lidar com dados não estruturados e escalabilidade horizontal. A partir de 2010, consolidou-se o conceito de polyglot persistence, que preconiza o uso do banco de dados mais adequado para cada tipo de problema dentro de uma mesma organização (Stonebraker; Çetintemel; Zdonik, 2005).

Em ambientes industriais modernos, os bancos de dados desempenham papel fundamental na integração de dados provenientes de diferentes sistemas organizacionais. Informações geradas por sensores industriais, sistemas de planejamento empresarial e plataformas de monitoramento da produção são armazenadas e organizadas em bancos de dados que permitem posterior análise e geração de indicadores de desempenho (Corallo et al., 2022).

As duas principais abordagens de armazenamento — relacional (SQL) e documental (NoSQL) — apresentam características distintas, sendo a escolha entre elas determinada pelo tipo de dado, volume, estrutura e requisitos de consistência do sistema. A Tabela 1 sintetiza as principais diferenças entre essas abordagens no contexto da Engenharia de Produção.

Tabela 1 — Comparativo entre bancos de dados relacionais e documentais (NoSQL).

Aspecto	Relacional (SQL)	Documental (NoSQL)
Unidade	Tabela (linhas e colunas)	Documento (JSON/BSON)
Esquema	Rígido, definido previamente	Flexível, schema-less
Relacionamentos	Chaves estrangeiras + JOINS	Documentos embutidos ou referências
Transações	ACID (consistência forte)	BASE (consistência eventual)

Escala	Vertical (hardware maior)	Horizontal (mais servidores)
Exemplo EP	ERP, estoque, financeiro	IoT, sensores, logs de produção

Fonte: adaptado de Silberschatz, Korth e Sudarshan (2020).

Com o avanço da transformação digital e da Indústria 4.0, o uso de técnicas de Big Data Analytics tem se tornado cada vez mais relevante para análise de grandes volumes de dados industriais. Essas técnicas permitem identificar padrões operacionais, prever falhas e otimizar processos produtivos, contribuindo para o aumento da eficiência organizacional (Wang et al., 2022). Além disso, novas tecnologias, como digital twins e plataformas de dados industriais, permitem integrar diferentes fontes de dados e ampliar as capacidades analíticas em ambientes produtivos (Zhang; Tao; Nee, 2024).

2.4 Modelagem de Dados e Normalização

A modelagem de dados é um processo fundamental no desenvolvimento de sistemas de informação, pois define a forma como os dados serão estruturados e organizados dentro de um banco de dados. De acordo com Elmasri e Navathe (2016), a modelagem de dados permite representar entidades do mundo real e seus relacionamentos, facilitando a organização e o armazenamento das informações.

Um dos principais conceitos associados à modelagem de dados é o processo de normalização, que consiste na organização das tabelas de forma a reduzir redundâncias e evitar inconsistências nos dados. Esse processo é estruturado em diferentes formas normais que orientam a construção de esquemas de banco de dados mais eficientes.

A primeira forma normal (1FN) estabelece que cada atributo de uma tabela deve conter valores atômicos, evitando estruturas repetitivas. A segunda forma normal (2FN) determina que todos os atributos não-chave dependam integralmente da chave primária da tabela. Já a terceira forma normal (3FN) elimina dependências transitivas entre atributos não-chave, promovendo maior consistência e integridade das informações armazenadas (Elmasri; Navathe, 2016).

Além da normalização, a modelagem de dados também envolve a definição de relacionamentos entre entidades, que podem ser classificados como um-para-um (1:1), um-para-muitos (1:N) ou muitos-para-muitos (N:M). Esses relacionamentos representam as

interações entre diferentes entidades do sistema e são essenciais para garantir a integridade referencial dos dados.

Dois conceitos fundamentais complementam a modelagem de relacionamentos: a agregação e a composição. A agregação representa uma relação do tipo "tem um", na qual as partes existem de forma independente do todo — por exemplo, um Fornecedor que se relaciona com um Pedido de Compra: o fornecedor continua existindo mesmo na ausência de pedidos. No contexto SQL, a agregação é implementada por meio de JOINS entre tabelas independentes. A composição, por sua vez, representa uma relação do tipo "parte de", na qual as partes não possuem existência autônoma sem o todo — por exemplo, os Itens de uma Ordem de Produção, que só existem vinculados a uma ordem específica. No SQL, a composição é implementada com a cláusula ON DELETE CASCADE na chave estrangeira da tabela filha (Elmasri; Navathe, 2016).

Com o avanço das tecnologias de análise de dados e inteligência artificial, a qualidade da modelagem de dados tornou-se ainda mais relevante. Estruturas de dados organizadas permitem que algoritmos analíticos e sistemas inteligentes utilizem essas informações de forma mais eficiente, ampliando as possibilidades de aplicação de técnicas de análise preditiva e otimização em ambientes industriais (Akram et al., 2026).

2.5 Entidades, Atributos e Transformação em Tabelas de Banco de Dados

No contexto da modelagem de dados relacional, o conceito de entidade designa qualquer objeto ou fenômeno do mundo real sobre o qual se deseja armazenar informações. Em sistemas produtivos, exemplos típicos de entidades incluem máquinas, operadores, ordens de produção, produtos acabados e centros de trabalho. Cada entidade é caracterizada por um conjunto de atributos, que são as propriedades ou características que descrevem aquela entidade de forma suficientemente precisa para os fins analíticos do sistema (Elmasri; Navathe, 2016).

A transformação de entidades em tabelas de banco de dados constitui a etapa central do processo de implementação do modelo relacional. Nessa transformação, cada entidade do modelo conceitual corresponde a uma tabela, cada atributo da entidade corresponde a uma coluna dessa tabela, e cada ocorrência individual da entidade, denominada instância, corresponde a uma linha (ou registro) da tabela. Para garantir a unicidade de cada registro, toda tabela deve possuir uma chave primária (*Primary Key*), que consiste em um atributo ou conjunto de atributos cujos valores identificam univocamente cada linha (Silberschatz; Korth; Sudarshan, 2020).

No ambiente da Engenharia de Produção, esse mapeamento pode ser ilustrado por meio de entidades recorrentes em sistemas de manufatura. A entidade MÁQUINA, por exemplo, pode ser representada pela tabela máquinas, contendo atributos como id_maquina (PK), codigo_patrimonio, descricao, centro_custo, data_aquisicao e status_operacional. De forma análoga, a entidade OPERADOR é representada pela tabela operadores, com atributos id_operador (PK), matrícula, nome, turno e id_setor. Já a entidade ORDEM DE PRODUÇÃO, central no controle do chão de fábrica, origina a tabela ordens_producao, com atributos id_ordem (PK), numero_op, id_produto, id_maquina (FK), id_operador (FK), data_abertura, data_fechamento e tempo_ciclo_minutos.

A inclusão das chaves estrangeiras (*Foreign Keys*) nesta última entidade evidencia a natureza relacional do modelo: os atributos id_maquina e id_operador não armazenam dados redundantes sobre a máquina ou o operador, mas sim referências às suas respectivas chaves primárias nas tabelas correspondentes. Essa abordagem garante a integridade referencial do banco de dados, impedindo, por exemplo, que uma ordem de produção faça referência a uma máquina inexistente no sistema (Date, 2004).

Os tipos de relacionamentos entre entidades influenciam diretamente a estrutura das tabelas. Em um relacionamento 1:1 (um-para-um), como entre FUNCIONÁRIO e CRACHÁ, a chave estrangeira pode ser inserida em qualquer uma das tabelas envolvidas. Em relacionamentos 1:N (um-para-muitos), como entre SETOR e FUNCIONÁRIOS, a chave estrangeira é inserida na tabela que representa o lado N da relação. Nos relacionamentos N:M (muitos-para-muitos), como entre PRODUTO e FORNECEDOR, é necessário criar uma tabela associativa que contém as chaves primárias das duas entidades envolvidas, podendo ainda carregar atributos próprios do relacionamento (Elmasri; Navathe, 2016).

A Tabela 2 sintetiza as principais entidades do cenário produtivo adotado neste trabalho, seus atributos principais e os tipos de relacionamentos que as conectam.

Tabela 2 — Entidades, tabelas e relacionamentos do cenário produtivo.

Entidade	Tabela	Atributos Principais	Relacionamentos
Máquina	maquinas	id_maquina (PK), codigo, descricao, status	1:N com ordens_producao
Operador	operadores	id_operador (PK), matricula, nome, turno	1:N com ordens_producao

Produto	produtos	id_produto (PK), código, descrição, família	1:N com ordens_producao
Ordem de Produção	ordens_producao	id_ordem (PK), id_maquina (FK), id_operador (FK), tempo_ciclo, qtd_produzida	N:1 com máquinas, operadores
Linha de Produção	linhas_producao	id_linha (PK), descrição, capacidade	1:N com maquinas

Fonte: autores.

2.6 UML — Unified Modeling Language na Modelagem de Sistemas e Dados

A Unified Modeling Language (UML) é uma linguagem de modelagem visual padronizada pela Object Management Group (OMG), amplamente utilizada para especificar, visualizar, construir e documentar estruturas e comportamentos de sistemas de software (Booch; Rumbaugh; Jacobson, 2005). Desde sua formalização em 1997, a UML consolidou-se como referência internacional para a comunicação entre engenheiros de software, analistas de sistemas e demais partes interessadas no desenvolvimento de sistemas de informação.

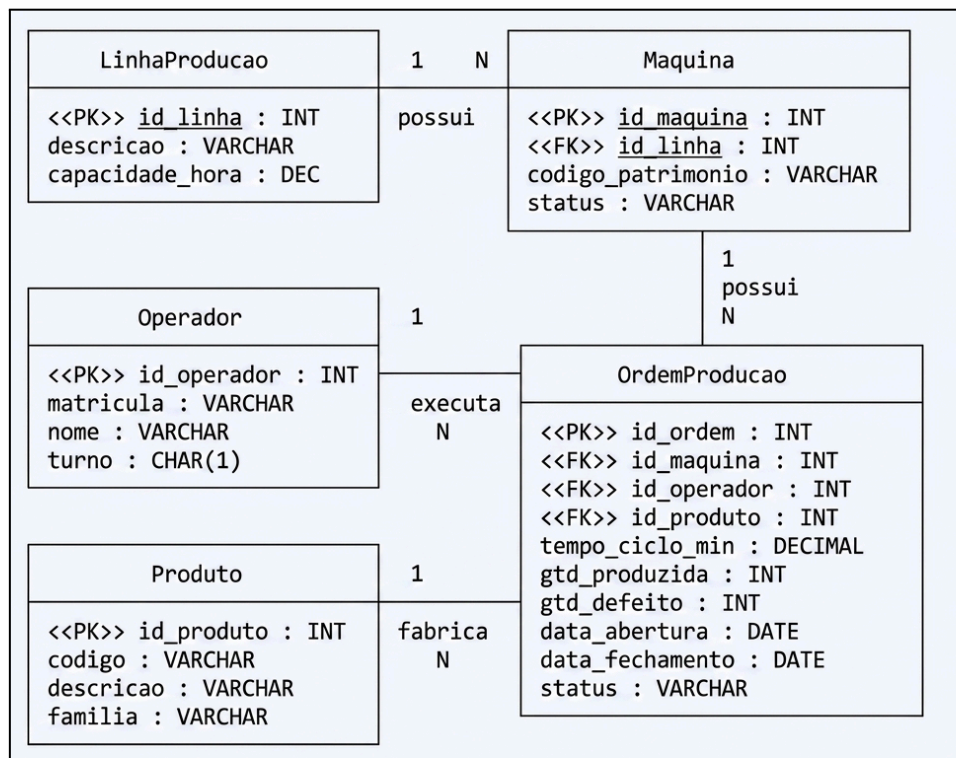
No contexto da modelagem de dados e sistemas de informação industrial, dois diagramas UML são de especial relevância: o Diagrama de Classes e o Diagrama Entidade-Relacionamento (DER). Embora possuam origens distintas, o primeiro proveniente da orientação a objetos e o segundo do modelo relacional proposto por Chen (1976), ambos exercem funções complementares na representação conceitual de sistemas de informação em Engenharia de Produção.

O Diagrama de Classes da UML representa as classes de um sistema, seus atributos, métodos e os relacionamentos entre elas. No âmbito da modelagem de dados, quando utilizado sem os métodos (operações), o Diagrama de Classes comporta-se de forma análoga a um diagrama estrutural que mapeia as entidades, seus atributos e as associações entre elas, incluindo multiplicidades (cardinalidades) nos extremos das linhas de associação. Essa notação permite representar relacionamentos 1:1, 1:N e N:M de forma precisa, além de capturar conceitos de herança, agregação e composição — distinções fundamentais em sistemas produtivos complexos (Fowler, 2003).

Como exemplo de um cenário produtivo neste trabalho, a construção do diagrama UML inicia-se pela identificação das entidades principais: MÁQUINA, OPERADOR, PRODUTO, ORDEM DE PRODUÇÃO e LINHA DE PRODUÇÃO. Cada entidade é

representada como uma classe com seus atributos tipados, sendo a chave primária indicada com o estereótipo <<PK>> e as chaves estrangeiras com <<FK>>. A Figura 1 apresenta o Diagrama de Classes UML para o cenário produtivo.

Figura 1 — Diagrama de Classes UML para o cenário produtivo de referência.



Fonte: autores.

2.7 Banco de Dados Não Relacional: MongoDB em Ambientes Industriais

Os bancos de dados não relacionais, amplamente conhecidos como bancos NoSQL, constituem uma categoria de sistemas de gerenciamento de dados que abandona a estrutura tabular rígida do modelo relacional em favor de modelos de armazenamento alternativos, projetados para atender a requisitos de escalabilidade horizontal, flexibilidade de esquema e alta disponibilidade (Stonebraker; Çetintemel; Zdonik, 2005). Diferentemente dos bancos relacionais, que organizam os dados em tabelas com linhas e colunas e exigem a definição prévia de um esquema fixo, os bancos NoSQL permitem o armazenamento de dados com estruturas variáveis, sem a obrigatoriedade de um modelo rígido predefinido. Essa característica torna os bancos NoSQL especialmente adequados para o tratamento de dados semiestruturados ou não estruturados, como os gerados por sensores industriais, sistemas de IoT, logs de máquinas e eventos de produção.

Entre as diferentes categorias de bancos NoSQL — documentais, de chave-valor, de colunas largas e de grafos —, o modelo documental é o mais amplamente adotado na indústria e o mais adequado para muitas aplicações de Engenharia de Produção. O MongoDB é o principal representante dessa categoria, sendo um sistema de gerenciamento de banco de dados documental de código aberto que armazena dados no formato BSON (Binary JSON), uma extensão binária do JSON. Cada registro no MongoDB é denominado documento, que é uma estrutura de dados flexível composta por pares de campo e valor. Os documentos são agrupados em coleções, que correspondem conceitualmente às tabelas do modelo relacional — porém sem a obrigatoriedade de que todos os documentos de uma mesma coleção compartilhem a mesma estrutura de campos (Chodorow, 2013).

A hierarquia de organização no MongoDB compreende três níveis: o banco de dados (database), que agrupa coleções; as coleções (collections), que agrupam documentos relacionados por finalidade; e os documentos (documents), que são as unidades fundamentais de armazenamento. Essa estrutura difere essencialmente do modelo relacional — no qual a hierarquia equivalente seria banco de dados → tabelas → linhas. A principal distinção reside no fato de que cada documento pode conter campos aninhados e arrays, permitindo representar estruturas complexas dentro de um único objeto, sem necessidade de múltiplas tabelas e operações de JOIN para reconstruir a informação completa (Chodorow, 2013).

Do ponto de vista das vantagens em relação ao banco relacional, o MongoDB oferece: esquema flexível (schema-less), que permite a evolução da estrutura de dados sem migrações complexas; escalabilidade horizontal nativa por meio de sharding; alta disponibilidade por meio de replica sets; e desempenho elevado em operações de leitura de documentos complexos. Em contrapartida, as limitações mais relevantes incluem: menor suporte nativo a transações entre múltiplas coleções; ausência de integridade referencial nativa; e maior consumo de espaço em disco, resultante da redundância intrínseca ao modelo de documentos embutidos (Silberschatz; Korth; Sudarshan, 2020).

No contexto da Engenharia de Produção e da Indústria 4.0, o MongoDB apresenta aplicabilidade particularmente relevante para o armazenamento de dados provenientes de fontes heterogêneas com estrutura variável. Sensores industriais que coletam diferentes conjuntos de variáveis conforme o tipo de equipamento, logs de eventos de máquinas e registros de qualidade com atributos específicos por família de produto são exemplos de cenários em que a flexibilidade documental supera as limitações do modelo tabular rígido (Wang et al., 2022). Nesse sentido, o uso combinado de PostgreSQL para dados transacionais estruturados e de MongoDB para dados operacionais semiestruturados — abordagem

conhecida como polyglot persistence — representa uma estratégia arquitetural robusta para ambientes industriais modernos.

Para fins de planejamento e referência metodológica, o Quadro 1 descreve um cenário hipotético de coleções MongoDB aplicadas a um ambiente de manufatura, com os tipos de informações que cada coleção poderia armazenar. A estrutura exata será definida a partir do banco de dados real a ser disponibilizado na N2.

Quadro 1 — Coleções MongoDB hipotéticas para um ambiente de manufatura.

Coleção	Informações armazenadas (hipotéticas)
maquinas	Dados cadastrais do equipamento: código, descrição, centro de custo, fabricante, data de aquisição, status operacional e especificações técnicas variáveis por tipo de máquina.
operadores	Dados dos colaboradores: matrícula, nome, turno, setor, qualificações. Campos opcionais variam por categoria funcional.
produtos	Informações sobre os itens fabricados: código, descrição, família, especificações técnicas e parâmetros de qualidade.
ordens_producao	Registros de fabricação: número da ordem, referências às coleções de produto, máquina e operador, datas, tempos de ciclo, quantidades produzidas e defeitos.
eventos_maquina	Log de eventos operacionais: tipo do evento (parada, alarme, setup, manutenção), instante do registro, duração e campos específicos por tipo de evento.
leituras_sensores	Séries temporais de variáveis físicas: temperatura, vibração, pressão, corrente elétrica. Alta frequência de escrita; estrutura de campos varia por tipo de sensor.
logs_qualidade	Registros de inspeção e controle de qualidade: resultado, valores medidos, especificações, operador responsável e campos por plano de controle.

Fonte: autores.

A seguir, apresentam-se exemplos hipotéticos de estruturas de documentos JSON para quatro coleções do cenário produtivo, com o objetivo de ilustrar a notação e a flexibilidade do modelo documental.

Exemplo 1 — Documento hipotético da coleção máquinas:

Documento 1 — Estrutura hipotética de um documento na coleção máquinas.

```
{
  "_id": ObjectId("..."),
  "codigo_patrimonio": "MAQ-001",
  "descricao": "Torno CNC Modelo X",
  "fabricante": "Empresa ABC",
  "data_aquisicao": ISODate("2020-03-15"),
  "status": "operacional",
  "centro_custo": "CC-03",
  "especificacoes": {
    "potencia_kw": 15,
    "velocidade_max_rpm": 3000,
    "eixos": 3
  }
}
```

Exemplo 2 — Documento hipotético da coleção ordens_producao:

Documento 2 — Estrutura hipotética de um documento na coleção ordens_producao.

```
{
  "_id": ObjectId("..."),
  "numero_op": "OP-2026-001",
  "status": "concluida",
  "produto_ref": ObjectId("..."),
  "maquina_ref": ObjectId("..."),
  "operador_ref": ObjectId("..."),
  "data_abertura": ISODate("2026-05-01T08:00:00Z"),
  "data_fechamento": ISODate("2026-05-01T10:30:00Z"),
  "tempo_ciclo_min": 150,
  "qtd_produzida": 45,
  "qtd_defeito": 2
}
```

Exemplo 3 — Documento hipotético da coleção leituras_sensores:

Documento 3 — Estrutura hipotética de um documento na coleção leituras_sensores.

```
{
  "_id": ObjectId("..."),
  "maquina_ref": ObjectId("..."),
  "timestamp": ISODate("2026-05-01T09:15:30Z"),
  "tipo_sensor": "temperatura",
  "leituras": {
    "temperatura_c": 72.4,
    "vibracao_mm_s": 1.8,
    "corrente_a": 12.3
  }
}
```

Exemplo 4 — Documento hipotético da coleção eventos_maquina:

Documento 4 — Estrutura hipotética de um documento na coleção eventos_maquina.

```
{
  "_id": ObjectId("..."),
  "maquina_ref": ObjectId("..."),
  "tipo_evento": "parada_nao_planejada",
  "timestamp_inicio": ISODate("2026-05-01T09:45:00Z"),
  "timestamp_fim": ISODate("2026-05-01T10:05:00Z"),
  "duracao_min": 20,
  "causa": "falha_rolamento",
  "operador_ref": ObjectId("...")
}
```

Com o objetivo de orientar a escolha do SGBD mais adequado a cada contexto analítico, a Tabela 2 apresenta um comparativo detalhado entre PostgreSQL e MongoDB no contexto da Engenharia de Produção.

Tabela 2 — Comparativo entre PostgreSQL e MongoDB no contexto da Engenharia de Produção.

Critério	PostgreSQL	MongoDB
Tipo de banco	Relacional (SQL)	Não relacional (NoSQL documental)
Estrutura de armazenamento	Tabelas com linhas e colunas; esquema rígido	Coleções de documentos JSON/BSON; esquema flexível
Linguagem de consulta	SQL (SELECT, JOIN, GROUP BY, WINDOW)	MQL — MongoDB Query Language; pipelines de agregação
Ferramenta gráfica	pgAdmin / DBeaver	MongoDB Compass / DBeaver
Biblioteca Python	psycopg2 / SQLAlchemy	pymongo
Porta padrão	5432	27017
Transações	ACID completo (múltiplas tabelas)	ACID multidocumento (v4.0+); BASE por padrão
Melhor aplicação	Dados transacionais estruturados: ERP, MES, ordens de produção	Dados semiestruturados: IoT, sensores, logs, eventos
Exemplo EP	Ordens de produção, indicadores OEE, controle de estoque	Leituras de sensores, logs de paradas, dados de manutenção preditiva

Fonte: autores, adaptado de Silberschatz, Korth e Sudarshan (2020) e Chodorow (2013).

3. Metodologia

3.1 Tipo de Pesquisa

O presente trabalho caracteriza-se como uma pesquisa de natureza aplicada, pois visa desenvolver conhecimento orientado à resolução de um problema prático e específico, a sistematização do processo de acesso, extração e análise de dados em bancos de dados industriais para fins de apoio à tomada de decisão em Engenharia de Produção (Gil, 2002). Do ponto de vista dos objetivos, a pesquisa é descritiva, uma vez que busca delinear características e etapas de um procedimento operacional, sem manipulação de variáveis ou controle experimental.

Quanto à abordagem, o trabalho combina elementos qualitativos e quantitativos. A dimensão qualitativa manifesta-se na análise da estrutura dos sistemas de dados, na elaboração do POP e na interpretação dos indicadores gerados, enquanto a dimensão quantitativa se expressa nas consultas SQL e nos cálculos de métricas de desempenho produtivo. Do ponto de vista dos procedimentos técnicos, trata-se de uma pesquisa de natureza documental e tecnológica, sustentada por revisão bibliográfica sistemática e pela proposição de um procedimento operacional fundamentado em evidências científicas e práticas de mercado (Prodanov; Freitas, 2013).

3.2 Procedimento Operacional Padrão — Banco de Dados Relacional (PostgreSQL)

O POP desenvolvido neste trabalho é estruturado em sete etapas sequenciais, cada uma com entradas, saídas e critérios de qualidade definidos. A seguir, cada etapa é detalhada com seus objetivos, ações e artefatos resultantes, apresentados no formato de Procedimento Operacional Padrão conforme as diretrizes da ISO 9001:2015.

Objetivo
Estabelecer um procedimento padronizado, reproduzível e rastreável para que o engenheiro de produção acesse, extraia e analise dados diretamente de bancos de dados industriais, gerando indicadores de desempenho que subsidiem a tomada de decisão baseada em evidências primárias.
Responsáveis

- **Engenheiro de Produção / Analista de Dados:** responsável pela execução do procedimento e documentação dos artefatos gerados em cada etapa.
- **Analista de TI / DBA:** responsável por fornecer credenciais de acesso, documentação técnica do banco e suporte à conexão.
- **Gestor de Operações:** responsável por validar o escopo das entidades mapeadas e aprovar formalmente o acesso ao banco de dados.

Recursos Necessários

- Credenciais de acesso ao banco de dados (host, porta, schema, usuário e senha) formalmente autorizadas pela área de TI;
- Ferramenta gráfica de acesso: DBeaver Community Edition ou pgAdmin;
- Python 3.11+ com as bibliotecas psycopg2, pandas e sqlalchemy instaladas;
- Documentação técnica do banco de dados (dicionário de dados ou schema físico);
- Autorização formal de acesso em conformidade com a LGPD (Lei nº 13.709/2018).

Procedimento

Etapa 1 — Identificação da Fonte de Dados

Objetivo: identificar e caracterizar a fonte de dados a ser utilizada, mapeando o sistema de origem, o tipo de banco e o escopo de dados disponíveis.

Como proceder:

1. Levantar junto à área de TI quais sistemas corporativos (ERP, MES, SCADA, sistema de qualidade) alimentam o banco e qual SGBD é utilizado.
2. Registrar os parâmetros de conexão: nome do SGBD, endereço do servidor (host), porta, nome do banco e schema principal.
3. Confirmar se o acesso é formalmente autorizado e se os dados pessoais eventualmente presentes estão anonimizados, conforme a LGPD.
4. Registrar a frequência de atualização dos dados (tempo real, diária, semanal) para avaliar a adequação ao tipo de análise pretendida.

Fontes primárias típicas em ambientes industriais: *ERP (Enterprise Resource Planning)*, *MES (Manufacturing Execution System)*, *SCADA (Supervisory Control and Data Acquisition)* e bases de dados específicas de qualidade, manutenção ou logística.

Resultado esperado: Documento de escopo contendo SGBD, parâmetros de conexão, sistema de origem e frequência de atualização dos dados.

Etapa 2 — Compreensão da Estrutura do Banco de Dados

Objetivo: explorar e documentar a estrutura lógica do banco de dados, identificando as tabelas relevantes, seus atributos, chaves e relacionamentos.

Como proceder:

1. Conectar ao banco pela ferramenta gráfica (DBeaver ou pgAdmin) e navegar pelos schemas disponíveis.
2. Identificar as tabelas com dados operacionais relevantes: ordens de produção, máquinas, operadores, produtos, histórico de qualidade e paradas.
3. Para cada tabela relevante, registrar: nome, descrição funcional, colunas principais com tipos de dados, chave primária (PK) e chaves estrangeiras (FK).
4. Estimar o volume de registros e o período histórico disponível, avaliando a suficiência dos dados para as análises planejadas.

Classificação dos relacionamentos entre tabelas:

- 1:1 (Um para Um) — cada registro de uma tabela relaciona-se com exatamente um da outra. Exemplo: FUNCIONÁRIO ↔ CRACHÁ.
- 1:N (Um para Muitos) — um registro de uma tabela relaciona-se com vários da outra. Exemplo: LINHA DE PRODUÇÃO → MÁQUINAS.
- N:M (Muitos para Muitos) — vários registros de uma tabela relacionam-se com vários da outra; requer tabela associativa. Exemplo: PRODUTO ↔ FORNECEDOR.

Resultado esperado: Glossário de dados listando cada tabela relevante, descrição, volume aproximado de registros e granularidade temporal (frequência de atualização).

Etapa 3 — Modelagem dos Dados

Objetivo: construir o modelo conceitual e lógico dos dados identificados, elaborando o Diagrama de Classes UML que servirá como mapa de referência para as consultas SQL.

Como proceder:

1. Com base nas tabelas e relacionamentos levantados na Etapa 2, identificar as entidades principais do cenário produtivo.
2. Para cada entidade, listar os atributos com seus tipos de dados e marcar a PK com o estereótipo <<PK>> e as FKs com <<FK>>.

3. Desenhar as linhas de associação entre as entidades, indicando a cardinalidade em cada extremo (1, N ou *).
4. Verificar se o banco atende às formas normais (1FN, 2FN, 3FN), registrando eventuais desnormalização que possam impactar as análises.

Caso o banco apresente desnormalização — prática comum em sistemas legados que sacrificam a 3FN em prol da performance de leitura —, o engenheiro deve registrar essa característica e tratá-la adequadamente nas queries, evitando dupla contagem ou distorção dos indicadores.

Resultado esperado: Diagrama de Classes UML documentado, validado com o gestor de operações e arquivado como referência para as etapas de extração.

Etapa 4 — Conexão com o Banco de Dados

Objetivo: estabelecer e validar a conexão com o banco de dados por meio de ferramenta gráfica e de código Python, garantindo reprodutibilidade e rastreabilidade do acesso.

Ferramentas recomendadas por SGBD:

Tabela 3— Combinações recomendadas de SGBD, ferramenta gráfica e biblioteca Python.

SGBD	Ferramenta Gráfica	Biblioteca Python	Porta Padrão
PostgreSQL	pgAdmin / DBeaver	psycopg2 / sqlalchemy	5432
MySQL / MariaDB	MySQL Workbench / DBeaver	mysql-connector-python	3306
Microsoft SQL Server	SQL Server Management Studio	pyodbc	1433
MongoDB	MongoDB Compass	pymongo	27017
SQLite	DB Browser for SQLite	sqlite3 (nativo)	—

Exemplo de conexão via Python (PostgreSQL + psycopg2):

Código 1 — Conexão ao PostgreSQL via psycopg2 com validação do acesso

```
import psycopg2
import pandas as pd

-- Parâmetros de conexão — use variáveis de ambiente em produção
conn = psycopg2.connect(
    host = 'servidor.empresa.com',
```

```
port    = 5432,
dbname = 'db_producao',
user    = 'eng_producao',
password = '*****'
)

-- Validar a conexão listando as tabelas disponíveis
cur = conn.cursor()
cur.execute("SELECT tablename FROM pg_tables WHERE schemaname = 'public'")
print('Tabelas disponíveis:', cur.fetchall())
conn.close()
```

Resultado esperado: Conexão estabelecida e validada via ferramenta gráfica e via Python, com o código documentado e versionado no repositório do projeto.

Etapa 5 — Extração de Dados via Consultas SQL

Objetivo: elaborar, executar e documentar as consultas SQL que extraíam os dados necessários para a análise, de forma incremental e progressivamente mais complexa.

Como proceder:

1. Listar as perguntas analíticas a serem respondidas e identificar, no Diagrama de Classes UML, as tabelas e campos necessários para cada uma.
2. Elaborar as queries em ordem crescente de complexidade: seleção simples → filtragem (WHERE) → agrupamento (GROUP BY) → junções (JOIN) → funções de janela (window functions).
3. Documentar cada query com um comentário explicativo descrevendo seu objetivo, as tabelas envolvidas e os indicadores produzidos.
4. Testar queries complexas com LIMIT 100 antes de executar sobre o conjunto completo, evitando impacto de performance no servidor.

Consultas SQL Planejadas (exemplificativas):

Consulta 1 — Tempo médio de ciclo por linha de produção:

Código 2 — Consulta SQL para tempo médio de ciclo por linha de produção

```
-- Objetivo: tempo médio e variabilidade do ciclo por linha de produção
-- Tabelas: ordens_producao, maquinas, linhas_producao
SELECT
    l.descricao                                AS linha_producao,
    COUNT(op.id_ordem)                         AS total_ordens,
    ROUND(AVG(op.tempo_ciclo_min), 2)          AS tempo_medio_min,
    ROUND(STDDEV(op.tempo_ciclo_min), 2)       AS desvio_padrao_min
FROM ordens_producao op
JOIN maquinas m      ON op.id_maquina = m.id_maquina
```

```
JOIN linhas_producao l ON m.id_linha = l.id_linha
WHERE op.status = 'concluida'
AND op.data_fechamento >= '2026-01-01'
GROUP BY l.descricao
ORDER BY tempo_medio_min DESC;
```

A consulta realiza dois JOINS em cadeia — entre ordens_producao e maquinas, e entre maquinas e linhas_producao — para navegar pelo relacionamento N:1:N que conecta ordens às máquinas e máquinas às linhas. O uso das funções de agregação AVG e STDDEV permite calcular, além da média, a variabilidade do tempo de ciclo, indicador relevante para análises de estabilidade do processo.

Consulta 2 — Índice de eficiência de qualidade por operador:

Código 3 — Consulta SQL para índice de eficiência de qualidade por operador.

```
-- Objetivo: índice de qualidade (% aprovados) por operador e turno
-- Tabelas: ordens_producao, operadores
SELECT
    o.nome AS operador,
    o.turno,
    SUM(op.qtd_produzida) AS
total_produzido,
    SUM(op.qtd_defeito) AS
total_defeitos,
    ROUND(
        (1.0 - SUM(op.qtd_defeito) / NULLIF(SUM(op.qtd_produzida), 0))
        * 100, 2) AS
indice_qualidade_pct
FROM ordens_producao op
JOIN operadores o ON op.id_operador = o.id_operador
WHERE op.status = 'concluida'
GROUP BY o.nome, o.turno
ORDER BY indice_qualidade_pct ASC;
```

Consulta 3 — Evolução mensal do volume de produção por família de produto:

Código 4 — Consulta SQL para evolução do volume de produção mensal por família de produtos.

```
-- Objetivo: série histórica mensal de produção por família de produto
-- Tabelas: ordens_producao, produtos
SELECT
    DATE_TRUNC('month', op.data_abertura) AS mes_referencia,
    p.familia AS familia_produto,
    COUNT(op.id_ordem) AS qtd_ordens,
    SUM(op.qtd_produzida) AS vol_total_produzido
FROM ordens_producao op
JOIN produtos p ON op.id_produto = p.id_produto
WHERE op.status = 'concluida'
GROUP BY mes_referencia, p.familia
ORDER BY mes_referencia, vol_total_produzido DESC;
```

Resultado esperado: Conjunto de consultas SQL documentadas, executadas e com resultados validados, prontas para carregamento em estruturas analíticas.

Etapa 6 — Organização dos Dados Extraídos

Objetivo: carregar os dados retornados pelas consultas SQL em estruturas analíticas, aplicar verificações de qualidade e realizar transformações necessárias para preparar os dados para análise.

Como proceder:

1. Carregar o resultado de cada query em um DataFrame pandas: `df = pd.read_sql(query, conn)`.
2. Verificar valores nulos — `df.isnull().sum()` — e investigar registros com campos críticos ausentes (ex.: `tempo_ciclo_min` nulo pode indicar ordem ainda em aberto).
3. Verificar tipos de dados — `df.dtypes` — garantindo que colunas de data estejam em formato `datetime` e não como `string`.
4. Identificar outliers: valores de `tempo_ciclo_min` muito acima do percentil 99 podem indicar ordens abertas por engano ou paradas não registradas.
5. Criar colunas derivadas quando necessário: ex., `tempo_ciclo_horas = df["tempo_ciclo_min"] / 60`.

Resultado esperado: DataFrame limpo e validado, com documentação das ocorrências de dados nulos, outliers tratados e transformações aplicadas, pronto para geração de indicadores.

Etapa 7 — Análise e Geração de Indicadores de Desempenho

Objetivo: aplicar métodos analíticos sobre os dados organizados para gerar indicadores de desempenho produtivo que suportem a tomada de decisão gerencial.

Principais indicadores de desempenho produtivo:

- Overall Equipment Effectiveness (OEE) — combina disponibilidade, performance e qualidade em um único índice de eficiência global do equipamento.
- Tempo Médio de Ciclo (TMC) por linha de produção — calculado como a média aritmética dos tempos de ciclo das ordens concluídas no período.
- Índice de Qualidade (IQ) por operador — razão entre itens aprovados e total produzido, multiplicada por cem.
- Lead Time Médio das Ordens de Produção — diferença entre data de fechamento e data de abertura, indicando a agilidade do processo produtivo.

- Volume de Produção Mensal por família de produto — permite identificar tendências sazonais e variações de demanda ao longo do tempo.

Como interpretar e apresentar os resultados:

1. Comparar cada indicador com a meta estabelecida pela organização ou com o desempenho histórico disponível no banco de dados.
2. Identificar desvios relevantes: um desvio-padrão elevado no tempo de ciclo de uma linha indica instabilidade do processo, não apenas desempenho médio inadequado.
3. Cruzar indicadores complementares para geração de hipóteses de causa-raiz, que serão investigadas via ferramentas clássicas da Engenharia de Produção (Diagrama de Ishikawa, análise dos 5 Porquês).
4. Elaborar relatório analítico com os indicadores gerados, as análises interpretativas e as recomendações para melhoria dos processos produtivos avaliados.

Resultado esperado: Relatório analítico com indicadores de desempenho produtivo, diagnóstico por linha/operador/produto e recomendações de ação gerencial fundamentadas em dados primários.

Observações Finais e Controles

- O acesso ao banco deve ser formalmente autorizado pela área de TI antes do início das atividades; credenciais nunca devem ser armazenadas em código-fonte.
- Toda query deve ser precedida de comentário explicativo com objetivo, tabelas envolvidas e indicadores produzidos.
- O Diagrama de Classes UML deve ser atualizado sempre que houver alterações estruturais no banco (novas tabelas, renomeações, depreciações).
- Todo tratamento de dados (exclusão de outliers, imputação de nulos) deve ser registrado no relatório com justificativa técnica.
- Os resultados devem ser validados pelo Coordenador de Produção antes de serem utilizados em decisões gerenciais.
- O procedimento completo deve ser revisado a cada seis meses ou quando houver mudanças significativas na estrutura do banco de dados ou nos processos produtivos.

3.3 Procedimento Operacional Padrão — Banco de Dados Não Relacional (MongoDB)

A seguir, apresenta-se a versão complementar do POP voltada ao banco de dados não relacional MongoDB. As sete etapas mantêm a mesma estrutura lógica do POP relacional, com as devidas adaptações às particularidades do modelo documental. O procedimento é

apresentado em caráter planejado, sendo sua execução efetiva prevista para a N2 após disponibilização do dataset.

Objetivo — POP MongoDB
Estabelecer um procedimento padronizado, reproduzível e rastreável para que o engenheiro de produção acesse, extraia e analise dados diretamente de bancos de dados não relacionais baseados em MongoDB, gerando indicadores a partir de dados semiestruturados de origem industrial — como leituras de sensores, logs de eventos, registros IoT e ordens de produção — que subsidiem a tomada de decisão baseada em evidências primárias.

Recursos Necessários — MongoDB
1. Credenciais de acesso ao banco MongoDB (host, porta, nome do banco, usuário e senha) formalmente autorizadas pela área de TI;
2. MongoDB Compass (ferramenta gráfica oficial) ou DBeaver com suporte a MongoDB;
3. Python 3.11+ com a biblioteca pymongo instalada;
4. Documentação das coleções disponíveis no banco (equivalente ao dicionário de dados);
5. Autorização formal de acesso em conformidade com a LGPD (Lei nº 13.709/2018).

Etapa 1 — Identificação da Fonte de Dados (MongoDB)
Objetivo: Identificar e caracterizar a fonte de dados não relacional a ser utilizada, verificando o tipo de dados armazenados (sensores, eventos, logs, ordens), a origem dos dados e o contexto industrial associado.
Como proceder:
1. Levantar junto à área de TI quais sistemas produtivos (plataformas IoT, sistemas SCADA, coletores de sensores, sistemas de manutenção preditiva) alimentam o banco MongoDB.

2.	Registrar os parâmetros de conexão: endereço do servidor (host), porta padrão (27017), nome do banco de dados e credenciais.
3.	Identificar as coleções presentes no banco e o tipo de dado predominante em cada uma (dados de sensores, eventos, ordens, logs).
4.	Registrar a frequência de atualização dos dados — muitos dados de sensores e IoT são gravados em alta frequência (segundos ou minutos) — para avaliar a adequação ao tipo de análise pretendida.
Resultado esperado: Documento de escopo contendo endereço do servidor, lista de coleções disponíveis, volume estimado de documentos e frequência de atualização dos dados.	

Etapa 2 — Compreensão da Estrutura do Banco de Dados (MongoDB)	
Objetivo: Explorar e documentar a estrutura das coleções do banco MongoDB, identificando os campos presentes nos documentos, os tipos de dados e as referências entre coleções.	
Como proceder:	
5.	Conectar ao banco pelo MongoDB Compass e explorar visualmente as coleções disponíveis.
6.	Para cada coleção relevante, inspecionar amostras de documentos para identificar os campos presentes, seus tipos de dados e a variabilidade estrutural entre documentos.
7.	Identificar campos que funcionam como referências entre coleções (equivalentes às chaves estrangeiras no modelo relacional), geralmente armazenados como ObjectId.
8.	Estimar o volume de documentos em cada coleção e o período histórico disponível, avaliando a suficiência para as análises planejadas.
Resultado esperado: Glossário de coleções descrevendo cada uma com seus campos principais, tipos de dados, volume de documentos e observações sobre variabilidade estrutural.	

Etapa 3 — Modelagem dos Dados (MongoDB)	
Objetivo: Construir o modelo conceitual das coleções identificadas, documentando as estruturas de documento, os campos principais e as referências entre coleções.	
Como proceder:	
9.	Com base nas coleções e campos levantados na Etapa 2, elaborar um esquema que represente as coleções e suas referências mútuas.
10.	Para cada coleção, documentar: nome, finalidade, campos principais com tipos de dados e campos que armazenam referências (ObjectId) a outras coleções.
11.	Identificar quais dados estão embutidos (embedded documents) e quais utilizam referências externas, registrando as implicações para a extração via pipeline de agregação.
12.	Avaliar se documentos com campos ausentes ou variáveis exigem tratamento especial nas etapas de extração e organização.
Resultado esperado: Mapa de coleções documentado com campos principais, referências entre coleções e observações sobre documentos embutidos e campos variáveis.	

Etapa 4 — Conexão com o Banco de Dados (MongoDB)	
Objetivo: Estabelecer e validar a conexão com o banco MongoDB por meio do MongoDB Compass e da biblioteca pymongo em Python, garantindo reprodutibilidade e rastreabilidade do acesso.	
Resultado esperado: Conexão estabelecida e validada via MongoDB Compass e via pymongo, com o código documentado e versionado no repositório do projeto.	

Código 5 — Conexão ao MongoDB via pymongo e listagem de coleções (planejado para execução na N2).

```
from pymongo import MongoClient
import pandas as pd

# Parâmetros de conexão – usar variáveis de ambiente em produção
client = MongoClient(
    host = "servidor.empresa.com",
    port = 27017,
    username = "<usuario>",
    password = "*****"
)

# Selecionar o banco de dados
db = client["<nome_do_banco>"]

# Listar as coleções disponíveis
print("Coleções disponíveis:", db.list_collection_names())

# Inspeccionar amostra de documentos de uma coleção
sample = list(db["<colecao>"].find().limit(3))
for doc in sample:
    print(doc)

client.close()
```

Etapa 5 — Extração de Dados via Consultas MongoDB (Planejada para N2)	
Objetivo: Elaborar e documentar as consultas e pipelines de agregação MongoDB que extrairão os dados necessários para análise. A execução efetiva ocorrerá na N2, após a disponibilização do banco real.	
Como se pretende proceder:	
13.	Listar as perguntas analíticas a serem respondidas e identificar, no mapa de coleções, quais campos e coleções são necessários para cada uma.
14.	Elaborar consultas em ordem crescente de complexidade: busca simples (find) → filtragem (\$match) → agrupamento (\$group) → referências entre coleções (\$lookup) → ordenação (\$sort).
15.	Documentar cada consulta com comentário descrevendo seu objetivo, as coleções envolvidas e os indicadores a serem produzidos.
16.	Testar pipelines complexos com \$limit antes de processar o conjunto completo, evitando impacto de performance no servidor.

Resultado esperado (N2): Conjunto de consultas e pipelines documentados, executados sobre o banco real e com resultados validados, prontos para carregamento em DataFrames pandas.

Exemplos de consultas MongoDB planejadas (hipotéticas — serão adaptadas ao banco real na N2):

Consulta MongoDB 1 — Buscar ordens concluídas e volume por produto:

Código 6 — Consulta pymongo para busca de ordens concluídas e volume produzido por produto (hipotético).

```
from pymongo import MongoClient
import pandas as pd

client = MongoClient("mongodb://<usuario>:<senha>@<host>:27017/")
db = client["<nome_do_banco>"]

# Buscar ordens de produção concluídas
ordens = list(db["ordens_producao"].find({"status": "concluida"}))
df_ordens = pd.DataFrame(ordens)

# Calcular volume produzido por produto via pipeline de agregação
pipeline_volume = [
    {"$match": {"status": "concluida"}},
    {"$group": {
        "_id": "$produto_ref",
        "total_produzido": {"$sum": "$qtd_produzida"},
        "total_defeitos": {"$sum": "$qtd_defeito"},
        "total_ordens": {"$sum": 1}
    }},
    {"$sort": {"total_produzido": -1}}
]
df_volume = pd.DataFrame(db["ordens_producao"].aggregate(pipeline_volume))
print(df_volume)
client.close()
```

Consulta MongoDB 2 — Temperatura média por máquina e eventos de parada:

Código 7 — Pipelines pymongo para temperatura média e identificação de paradas (hipotético).

```
# Calcular média de temperatura por máquina
pipeline_temp = [
    {"$match": {"tipo_sensor": "temperatura"}},
    {"$group": {
        "_id": "$maquina_ref",
        "temp_media": {"$avg": "$leituras.temperatura_c"},
        "total_leituras": {"$sum": 1}
    }},
    {"$sort": {"temp_media": -1}}
]
df_temp = pd.DataFrame(db["leituras_sensores"].aggregate(pipeline_temp))

# Identificar eventos de parada não planejada
pipeline_paradas = [
    {"$match": {"tipo_evento": "parada_nao_planejada"}},
    {"$group": {
        "_id": "$maquina_ref",
        "total_paradas": {"$sum": 1},
        "duracao_total": {"$sum": "$duracao_min"}
    }},
    {"$sort": {"duracao_total": -1}}
]
df_paradas = pd.DataFrame(db["eventos_maquina"].aggregate(pipeline_paradas))
```

Consulta MongoDB 3 — Série histórica de produção por mês:

Código 8 — Pipeline MongoDB para série histórica mensal de produção (hipotético).

```
# Agregar produção por mês
pipeline_mensal = [
    {"$match": {"status": "concluida"}},
    {"$group": {
        "_id": {"mes": {"$dateToString": {
            "format": "%Y-%m", "date": "$data_abertura"
        }}},
        "qtd_ordens": {"$sum": 1},
        "vol_produzido": {"$sum": "$qtd_produzida"},
        "total defeitos": {"$sum": "$qtd_defeito"}
    }},
    {"$sort": {"_id.mes": 1}}
]
df_mensal = pd.DataFrame(db["ordens_producao"].aggregate(pipeline_mensal))
print(df_mensal)
```

Etapa 6 — Organização dos Dados Extraídos — MongoDB (Planejada para N2)

Objetivo: Converter os resultados dos pipelines de agregação em DataFrames pandas, aplicar verificações de qualidade e realizar as transformações necessárias para análise de indicadores.

17.	Converter os resultados dos pipelines em DataFrames: <code>df = pd.DataFrame(list(db["colecacao"].aggregate(pipeline)))</code> .
18.	Tratar campos de data: converter strings de data para o tipo datetime com <code>pd.to_datetime()</code> .
19.	Verificar a presença de campos ausentes (None/NaN), frequentes em coleções com esquema flexível, e tratar conforme o contexto analítico.
20.	Expandir subdocumentos aninhados com <code>pd.json_normalize()</code> quando necessário.
21.	Identificar e documentar outliers em métricas contínuas (temperatura, tempo de ciclo, duração de paradas).
Resultado esperado (N2): DataFrames limpos e validados, com documentação das ocorrências de campos ausentes, outliers tratados e transformações aplicadas, prontos para geração de indicadores.	

Etapas 7 — Análise e Geração de Indicadores de Desempenho — MongoDB (Planejada para N2)	
Objetivo: Aplicar métodos analíticos sobre os dados extraídos do MongoDB para gerar indicadores de desempenho com foco em dados operacionais de sensores, eventos e logs industriais.	
Indicadores específicos planejados para dados MongoDB:	
22.	Disponibilidade de máquina — razão entre o tempo produtivo e o tempo total disponível, calculada a partir da coleção de eventos de parada.
23.	Temperatura média e variabilidade por máquina — extraídas da coleção de leituras de sensores para suporte à manutenção preditiva.
24.	Frequência e duração de paradas não planejadas — calculadas a partir da coleção de eventos de máquina, por tipo de causa.
25.	Volume de produção por período — agregação mensal ou semanal a partir das ordens de produção registradas no MongoDB.

26. Índice de qualidade por produto — razão entre quantidade aprovada e total produzido, integrado com registros de logs de qualidade.

3.4 Tecnologias e Ferramentas Adotadas

As tecnologias selecionadas para este trabalho foram escolhidas com base em sua aderência ao contexto industrial, disponibilidade de código aberto e ampla adoção no mercado. O SGBD adotado como referência é o PostgreSQL, em sua versão 16, por ser um banco de dados relacional de código aberto, com suporte robusto ao padrão SQL, alta confiabilidade, suporte a transações ACID e ampla utilização em sistemas ERP e MES industriais (Silberschatz; Korth; Sudarshan, 2020).

Como ferramenta gráfica de conexão e exploração, utiliza-se o DBeaver Community Edition, por ser uma ferramenta multiplataforma e multiSGBD que permite execução de queries, visualização de schemas, exportação de dados e geração automática de diagramas de entidade-relacionamento. Para as análises programáticas, a linguagem Python 3.11 é utilizada em conjunto com as bibliotecas psycopg2 (conexão ao PostgreSQL), pandas (manipulação de DataFrames), matplotlib e seaborn (visualizações gráficas) e SQLAlchemy (abstração de conexão e ORM opcional).

Tabela 4 — Tecnologias e ferramentas adotadas para cada tipo de banco de dados.

Componente	PostgreSQL (Relacional)	MongoDB (Não Relacional)
SGBD	PostgreSQL 16	MongoDB 7
Ferramenta gráfica	pgAdmin / DBeaver	MongoDB Compass / DBeaver
Biblioteca Python	psycopg2 / SQLAlchemy	pymongo
Porta padrão	5432	27017
Linguagem de consulta	SQL	MQL / Pipeline de agregação
Melhor aplicação EP	Dados transacionais estruturados (ordens, estoque, KPIs)	Dados semiestruturados (sensores, IoT, logs, eventos)

Fonte: autores.

REFERÊNCIAS BIBLIOGRÁFICAS

- AKRAM, D. K.; ADDULA, S. R.; LIND, M.; BROWN, S.; ODION, S. AI-driven hybrid deep learning and swarm intelligence for predictive maintenance of smart manufacturing robots in Industry 4.0. *Electronics*, v. 15, n. 3, 2026. DOI: <https://doi.org/10.3390/electronics15030715>.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software architecture in practice*. 3. ed. Boston: Addison-Wesley, 2013.
- BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *UML: guia do usuário*. 2. ed. Rio de Janeiro: Elsevier, 2005.
- CHEN, P. P. The entity-relationship model — toward a unified view of data. *ACM Transactions on Database Systems*, v. 1, n. 1, p. 9–36, 1976.
- CODD, E. F. A relational model of data for large shared data banks. *Communications of the ACM*, v. 13, n. 6, p. 377–387, 1970.
- CORALLO, A.; CRESPIANO, A. M.; LAZOI, M.; LEZZI, M. Model-based big data analytics-as-a-service framework in smart manufacturing. *Robotics and Computer-Integrated Manufacturing*, v. 76, 2022.
- DATE, C. J. *Introdução a sistemas de bancos de dados*. 8. ed. Rio de Janeiro: Elsevier, 2004.
- DAVENPORT, T. H. *Big data at work: dispelling the myths, uncovering the opportunities*. Boston: Harvard Business Review Press, 2014.
- ELMASRI, R.; NAVATHE, S. *Fundamentals of database systems*. 7. ed. Boston: Pearson, 2016.
- FOWLER, M. *UML distilled: a brief guide to the standard object modeling language*. 3. ed. Boston: Addison-Wesley, 2003.
- GIL, A. C. *Como elaborar projetos de pesquisa*. 4. ed. São Paulo: Atlas, 2002.
- ISO 9001:2015. *Sistemas de gestão da qualidade — Requisitos*. Genebra: International Organization for Standardization, 2015.
- JAVOID, M.; HALEEM, A.; SINGH, R.; SULTAN, S. Smart manufacturing and Industry 4.0 technologies: a review. *Sustainable Operations and Computers*, v. 5, 2024.
- LAUDON, K. C.; LAUDON, J. P. *Management information systems: managing the digital firm*. 16. ed. New York: Pearson, 2020.
- MARTIN, R. C. *Clean architecture: a craftsman's guide to software structure and design*. Boston: Prentice Hall, 2017.
- O'BRIEN, J. A.; MARAKAS, G. M. *Management information systems*. 10. ed. New York: McGraw-Hill, 2013.
- PIISPANEN, V. V.; HENTUNEN, P. Strategic integration of digitalization, Industry 4.0 and sustainability. *Circular Economy and Sustainability*, 2026.
- PRESSMAN, R. S.; MAXIM, B. R. *Software engineering: a practitioner's approach*. 9. ed. New York: McGraw-Hill, 2020.
- PRODANOV, C. C.; FREITAS, E. C. *Metodologia do trabalho científico*. 2. ed. Novo Hamburgo: Feevale, 2013.
- SENGE, P. M. *A quinta disciplina: arte e prática da organização que aprende*. São Paulo: Best Seller, 1990.
- SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Database system concepts*. 7. ed. New York: McGraw-Hill, 2020.
- SOMMERVILLE, I. *Software engineering*. 10. ed. Boston: Pearson, 2019.

STONEBRAKER, M.; ÇETINTEMEL, U.; ZDONIK, S. The 8 requirements of real-time stream processing. ACM SIGMOD Record, v. 34, n. 4, p. 42–47, 2005.

WANG, J.; XU, C.; ZHANG, J.; ZHONG, R. Y. Big data analytics for intelligent manufacturing systems: a review. Journal of Manufacturing Systems, v. 62, p. 738–752, 2022.

XU, L.; XU, E.; LI, L. Industry 4.0: state of the art and future trends. International Journal of Production Research, 2024.

ZHANG, Y.; TAO, F.; NEE, A. Y. C. Digital twin-driven smart manufacturing. Robotics and Computer-Integrated Manufacturing, 2024.